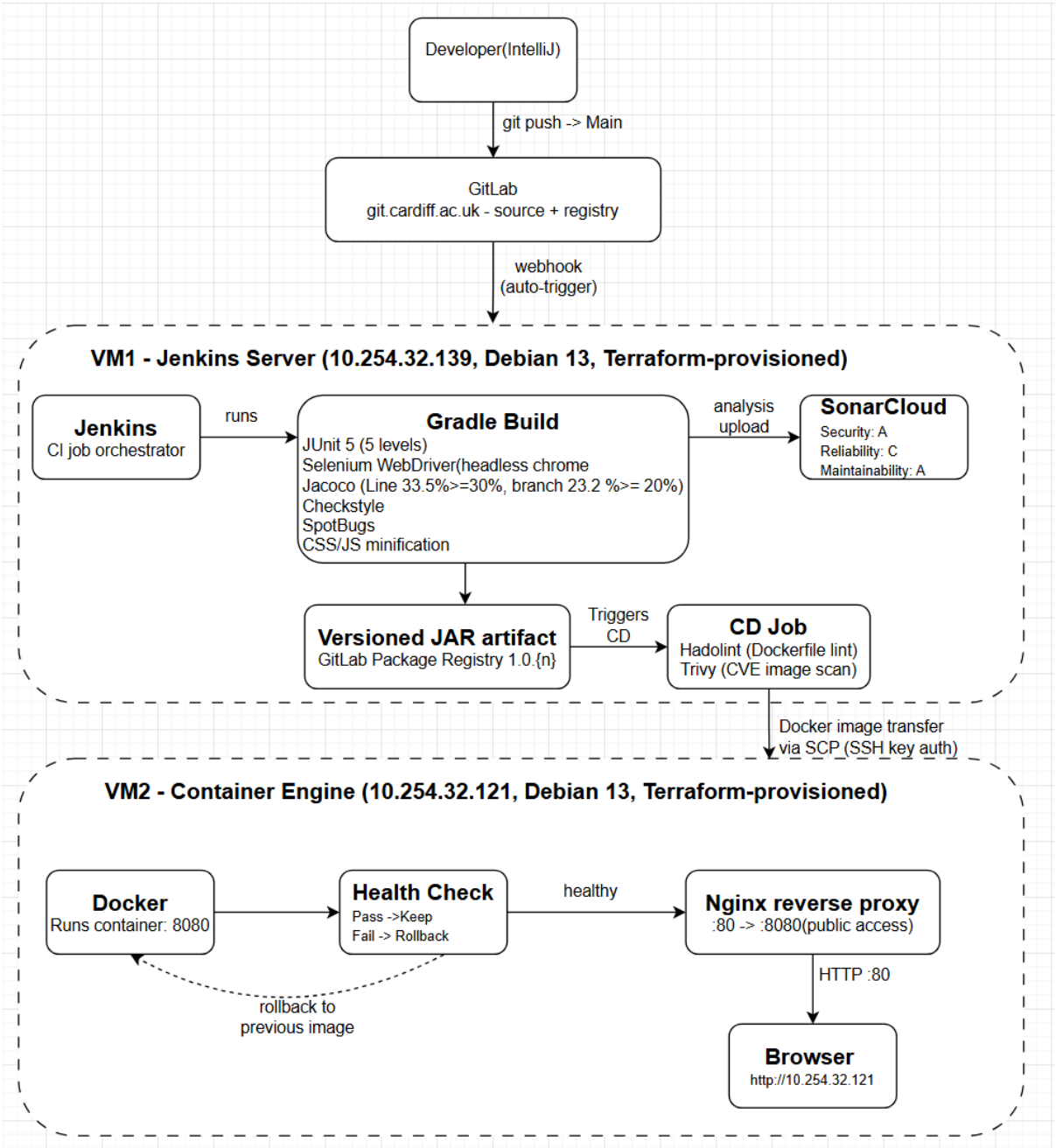
1. Pipeline Overview(1141 words)

The pipeline automates build, testing, quality analysis, and deployment of a Spring Boot application across two Terraform-provisioned Debian 13 VMs on Azure DTL. A GitLab webhook triggers Jenkins (VM1) on push, running a Gradle build with JUnit 5, Selenium WebDriver(headless Chrome), JaCoCo, Checkstyle, and SpotBugs; results are uploaded to SonarCloud. The JAR is versioned and published to the GitLab Package Registry, triggering CD: Hadolint and Trivy scan the Docker image before it's transferred to VM2 via SCP. Deployment is health-checked behind Nginx (:80 → :8080), with automatic rollback on failure.



Environment Decisions

1. Debian 13 Over AlmaLinux9

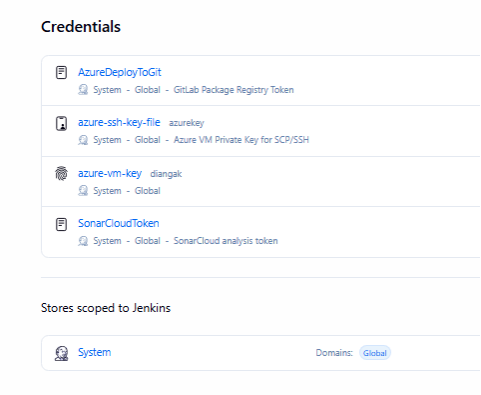
Debian 13(Trixie) was selected for all VMs, with OpenJDK 21 [1], MariaDB [2], Jenkins, and Docker installed via apt-get from official repositories. Jenkins[15] and Docker[16] are provisioned with GPG verification (`gpg --dearmor`) embedded in jenkins_setup.sh, ensuring deterministic, auditable provisioning across Terraform-managed VMs and preventing configuration drift between VM1 (CI) and VM2 (runtime). Host-container parity is maintained, as the application uses an Eclipse Temurin JRE (Debian-slim) base image [4].

AlmaLinux 9 was rejected due to its state-dependent setup (`dnf module enable java:21`) [3], which increases provisioning complexity and drift risk.

2. Two-VM split with SSH key bridge

The architecture separates CI (VM1) and runtime (VM2), prioritising security, fault isolation, and operational resilience over cost. Jenkins and build tools are isolated on VM1, while VM2 runs only Docker and Nginx, minimising installed components to reduce the attack surface and aligning with container security best practices [5]. This supports defence-in-depth by limiting lateral movement between systems [6].

Secure communication is implemented via SSH keys configured in `jenkins_setup.sh` and stored in the Jenkins Credentials Store, enabling authenticated image transfer without hardcoded secrets.



This separation improves reliability: build failures do not impact the live application, and runtime issues do not disrupt CI. Separating environments across two Terraform-provisioned VMs prevents configuration drift and aligns with IaC principles of independently deployable, reproducible infrastructure [7].

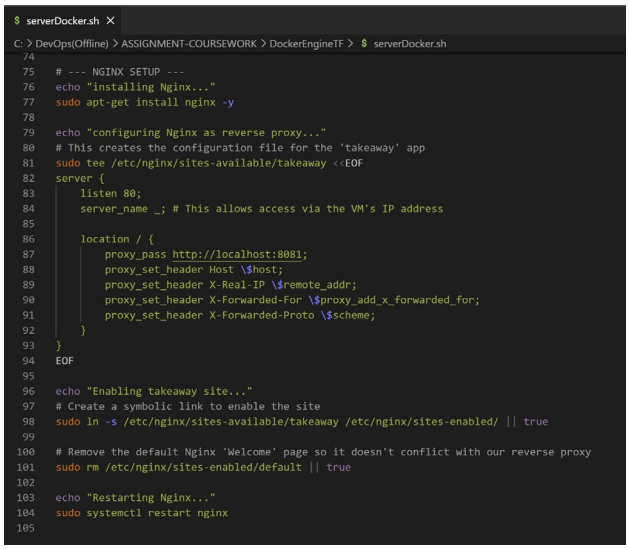
The trade-off is increased cost and configuration complexity, justified by stronger security isolation and production-aligned design. A limitation is brief downtime during `docker stop && docker run` deployments, which can be eliminated using a blue-green deployment behind Nginx with health-check-based upstream switching[4]. The current use of Bookworm repository in Docker installation is a temporary workaround and should be updated to Trixie in jenkins_setup.sh once officially supported to improve maintainability and consistency.

3. Nginx Reverse Proxy

Nginx is configured as a reverse proxy in `serverDocker.sh`, forwarding HTTP traffic from port 80 to `localhost:8081` via `proxy_pass`[8][17]. Forwarded headers (`X-Real-IP`, `X-Forwarded-For`, `X-Forwarded-Proto`) preserve client context. Docker maps host port 8081 to container port 8080, creating a layered request flow: browser → Nginx (:80) → host (:8081) → container (:8080).

This setup hides internal services and decouples the public interface from the application, improving security and flexibility. It also enables production features such as TLS termination, caching, and load balancing without modifying the application [9][10].

The application is accessible at `http://10.254.32.121` (port 80 via Nginx) and directly at `http://10.254.32.121:8081` (Docker host port) confirming correct mapping. Direct access to port 8081 bypasses the proxy; in production, firewall rules (`ufw deny 8081`) should enforce all traffic through Nginx to support controls like SSL and rate limiting.



4. GitLab Package Registry over local artifact storage.

JARs are versioned as 1.0.{BUILD_NUMBER} and published on every successful CI run, creating an immutable audit trail that enables redeployment of any prior version. Local storage on VM1 would be lost on re-provisioning.

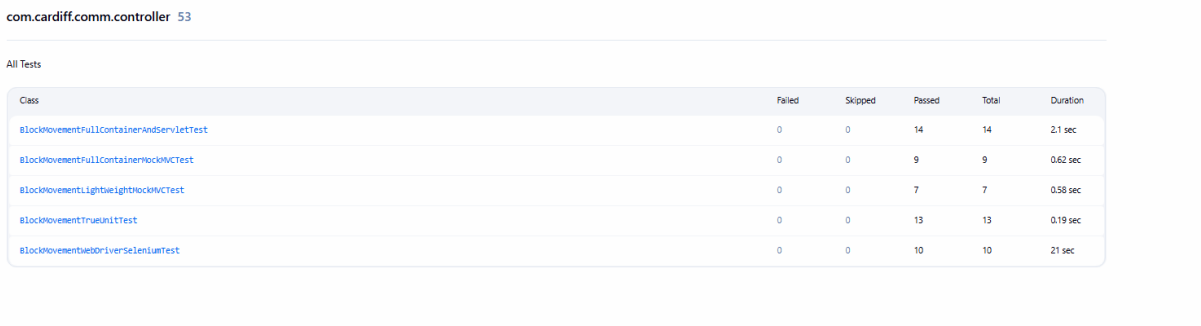
Testing Schedule

The testing strategy follows the Test Pyramid [11], informed by Agile ALM principles [14], and is implemented across five levels for the *BlockMovementController*. It prioritises fast feedback, reliability, and efficient CI execution over E2E-heavy approaches.

At the base (Q1), unit tests (Mockito) validate isolated logic and exception handling (e.g., 200/400/500 paths). MockMVC slice and full-context tests verify HTTP routing and controller–service–repository integration, aligning with CI best practices for early defect detection [12].

At the functional level (Q2), servlet tests confirm real HTTP behaviour, while Selenium WebDriver E2E tests validate Thymeleaf rendering and client-side interactions not observable server-side. UI tests are intentionally limited to critical flows to reduce brittleness and runtime overhead [14]. Chrome is provisioned via jenkins_setup.sh for consistent CI execution and selected over Firefox due to its official Debian package and GPG-verified installation [13].

The suite includes 53 tests, executing in ~21 seconds. JaCoCo enforces ≥30% line (33.54%) and ≥20% branch (23.18%) coverage across builds.



com.cardiff.comm.controller 53

All Tests

Class	Failed	Skipped	Passed	Total	Duration
BlockMovementFullContainerAndServletTest	0	0	14	14	2.1 sec
BlockMovementFullContainerMockMVCTest	0	0	9	9	0.62 sec
BlockMovementLightweightMockMVCTest	0	0	7	7	0.58 sec
BlockMovementTrueUnitTest	0	0	13	13	0.19 sec
BlockMovementWebDriverSeleniumTest	0	0	10	10	21 sec

Coverage is constrained by the view-heavy architecture; expanding controller contract/API tests would improve coverage while maintaining fast feedback cycles.

Tool Selection

Tool	Justification	Alternative	Limitation	Improvement
JUnit 5	Native Gradle and @SpringBootTest; supports all five test pyramid levels	TestNG: requires extra configuration for Spring Boot integration; added complexity unjustified for test suite size	Less advanced parameterisation	Use `@ParameterizedTest` where needed
Selenium WebDriver	Headless Chrome E2E; validates Thymeleaf and JS events not observable server-side	*Selenium IDE:* no headless/CI support; tests cannot be version-controlled or run non-interactively	Slower/more brittle than low-level tests	Restrict to critical paths in CI
JaCoCo	Java 21 compatible; Gradle-integrated; feeds Jenkins and SonarCloud	*Cobertura:* unmaintained, incompatible with Java 21 used in this project	33.54% line, 23.18% branch coverage	Spring REST Docs controller contract tests to push line coverage above 60%.
Checkstyle + SpotBugs	Source + bytecode analysis; complementary analysis layers that catch different defect classes	*PMD:* overlaps with SpotBugs producing redundant findings and adds build time with no benefit	Report-only quality gates, all tools run with `ignoreFailures = true`	Enforce staged build failures (e.g., fail on high severity)
SonarCloud	Aggregates JaCoCo, Checkstyle, SpotBugs into a unified dashboard with no extra VM required	*SonarQube*: requires third VM; high overhead for scale	Reduced control over data residency	In regulated environments, self-hosting would be required
Hadolint	Automated Dockerfile linting pre-build shifts security checks left,	Manual review: non-repeatable, non-auditable and	DL3018 technical debt	Pin eclipse-temurin:21-jre-alpine3.19 and an

	catching misconfigurations before image build time rather than post-deployment	cannot be integrated into an automated pipeline		Alpine package lockfile.
Trivy	Scans OS layers and Java JAR dependencies; detected 8 CVEs (5 CRITICAL Thymeleaf, 3 HIGH Tomcat)	*Clair:* OS-layer scanning only; misses JAR vulnerabilities	Does not fail builds	enforce `--exit-code 1` on CRITICAL/HIGH
CSS/JS Minification (Gradle task)	Gradle-integrated; runs pre-bootJar; ~23% size reduction	Node toolchain -adds runtime and build complexity	Not applied in JAR	Fix resource override in pipeline

Reflection on Week 11 Group Exercise

During the Week 11 exercise, the pipeline supported multi-user collaboration: a second Jenkins user and shared GitLab repo ensured all commits and merge requests triggered builds via webhook, with changes consistently reflected across users and Jenkins, confirming reliability.

A limitation was brief downtime during container replacement; this can be resolved using blue–green deployment behind Nginx with health-check-based switching to ensure continuous availability[18]. Furthermore, the reliance on GitLab push events to trigger Jenkins introduces a dependency on the webhook's stability and requires the Jenkins server to be actively maintained to prevent pipeline failures.

References

[1] Debian (2024a) *Package: openjdk-21-jdk (21.0.11+10-1~deb13u2) in trixie*. Available at: https://packages.debian.org/trixie/openjdk-21-jdk (Accessed: 2 May 2026).

[2] Debian (2024) *Package: mariadb-server in trixie*. Available at: https://packages.debian.org/trixie/mariadb-server (Accessed: 2 May 2026).

[3] Red Hat (2024) *Managing software with the DNF tool: Chapter 2. Distribution of content in RHEL 9*. Available at: https://docs.redhat.com/en/documentation/red_hat_enterprise_linux/9/html/managing_software_with_the_dnf_tool/assembly_distribution-of-content-in-rhel-9_managing-software-with-the-dnf-tool (Accessed: 2 May 2026).

[4] Humble, J. and Farley, D. (2010) *Continuous Delivery: Reliable Software Releases through Build, Test, and Deployment Automation*. Upper Saddle River, NJ: Addison-Wesley Professional.

[5] Rice, L. (2023) *Container Security: Fundamental Technology Concepts that Protect Cloud Native Applications*. 2nd edn. [Online]. Sebastopol, CA: O'Reilly Media. Available at: [7. Supply Chain Security | Container Security, 2nd Edition] (Accessed: 3 May 2026).

[6] Halley, J., Prajapati, D., Leza, A. and Saini, V. (2023) *Zero Trust in Resilient Cloud and Network Architectures*. [Online]. Indianapolis, IN: Cisco Press. Available at: [Chapter 19. Workload Mobility: On-Prem to Cloud | Zero Trust in Resilient Cloud and Network Architectures] (Accessed: 3 May 2026).

[7] Brikman, Y. (2022) *Fundamentals of DevOps and Software Delivery*. [Online]. Sebastopol, CA: O'Reilly Media. Available at: [2. How to Manage Your Infrastructure as Code | Fundamentals of DevOps and Software Delivery] (Accessed: 3 May 2026).

[8] NGINX (2026) *NGINX Reverse Proxy*. [Online]. Available at: https://docs.nginx.com/nginx/admin-guide/web-server/reverse-proxy/ (Accessed: 3 May 2026)

[9] Gavrilin, A., Ostrowski, A. and Gaczkowski, P. (2024) *Software Architecture with C++*. 2nd edn. [Online].Birmingham: Packt Publishing.Available at: Cloud-Native Design | Software Architecture with C++ - Second Edition (Accessed: 3 May 2026).

[10] Kumar, J. and Singh, M. (2023) *System Design on AWS*. [Online]. Sebastopol, CA: O'Reilly Media. Available at: 5. Load Balancing Approaches and Techniques | System Design on AWS (Accessed: 3 May 2026).

[11] Richardson, C. 2018. Microservices Patterns [Online]. Chapter 9: Testing Microservices: Part 1. Shelter Island, NY: Manning Publications. Available at: https://learning.oreilly.com/ library/view/microservices-patterns/9781617294549/OEBPS/Text/09.html [Accessed 3 May 2026].

[12] Durkin, N., Minick, E. and Gaikwad, C. 2024. AI-Native Software Delivery [Online]. Chapter 3: The Build and Pre-Deployment Testing Steps of Continuous Integration. Sebastopol, CA: O'Reilly Media. Available at: https://learning.oreilly.com/library/view/ ai-native-software-delivery/9781098171988/ch03.html [Accessed 3 May 2026].

[13] Selenium Project. 2024. Install a Selenium Library [Online]. Selenium WebDriver Documentation. Available at: https://www.selenium.dev/documentation/webdriver/getting_started/ install_library/ [Accessed 3 May 2026].

[14] Aiello, R. and Huettermann, M. 2010. Agile ALM [Online]. Chapter 8: Requirements and Test Management. Shelter Island, NY: Manning Publications. Available at: Chapter 8. Requirements and test management | Agile ALM [Accessed 3 May 2026].

[15] Docker Inc. 2024. Install Docker Engine on Debian [Online]. Available at: https://docs.docker.com/engine/install/debian/ [Accessed 3 May 2026].

[16] Jenkins. 2024. Debian Jenkins Packages [Online]. Available at: https://pkg.jenkins.io/debian-stable/ [Accessed 3 May 2026].

[17] NGINX Inc. 2024. nginx: Linux Packages — Debian [Online]. Available at: https://nginx.org/en/linux_packages.html#Debian [Accessed 3 May 2026].

[18] Ramer Labs. 2025. *The Ultimate Checklist for Zero-Downtime Deploys with Docker and Nginx*. Available at: The Ultimate Checklist for Zero-Downtime Deploys with Docker and Nginx - DEV Community [Accessed: 63May 2026].